Detecting phishing from trusted accounts

A layered, incident-driven anti-phishing toolkit: SentinelQA, TrustGuard and netguard

Technical write-up, release 1.1.0 (November 2026). The incident is anonymised; all test messages, names and ids are fictional. Every number here comes from the code and tests in this repository.

---

1. Summary

Most anti-phishing tools ask *is this link bad?* The incident behind this project needed a different question: *is this sender still who they seem to be?* An attacker took over a colleague's chat account and sent a short gift-card lure to all its contacts. Because the sender was familiar, several people typed their credentials into a fake Steam page, and anyone who reused that password lost other accounts too. When the hijacked account was removed, the attacker came back with a new one, and nothing asked who they were.

The project answers each step of that chain with its own, explainable control:

- **SentinelQA**, a Chrome (Manifest V3) extension, grades the page in front of the user from A to F. It catches look-alike brand domains, fake sign-in pages, rejected certificates and DNS oddities.
- **TrustGuard**, a chat bot with one platform-neutral engine and thin Discord and Slack adapters, scores messages from *trusted but compromised* accounts. It spots one sender pushing the same lure to many people, turns user reports of private DMs into moderator alerts, and holds returning or look-alike accounts for re-verification.
- **netguard**, a small command-line check, looks for signs of Wi-Fi interception: a swapped router, redirected DNS, rejected certificates.
- A **shared rule set**, one file of brand domains, abused domain endings and look-alike character pairs, keeps the extension and the bot in step.

Given the incident's lure, the extension grades `steamcommunnity.com/gift-card/pay/50` F (70/100), and the bot scores the same text 95/100 and quarantines it. Ordinary colleague messages, even one mentioning Steam, score 0. Everything except the Slack bot has been tested by hand as well as by 83 automated tests. The scoring is heuristic, and its accuracy on real traffic has not been measured (§11–12).

2. The incident

Reconstructed and anonymised from the author's own observation:

1. **Network.** A colleague used public Wi-Fi while travelling. Days later his Discord account was hijacked, most likely with a session token or password stolen on the open network and used later.
2. **Spread.** The attacker sent everyone on the friend list a "Steam gift $50" link styled like `steamcommunity.com/gift-card/pay/50`, with no greeting and nothing personal.
3. **Lure and harvest.** Trusting the name and tempted by the money, several recipients entered their Steam credentials on a look-alike page.
4. **Cascade.** Victims who reused the same email and password lost other accounts too.
5. **Re-infiltration.** The company removed the hijacked account from the work server. The attacker rejoined with a *new* account, and with no re-verification step, got in.

6. **Human factor.** At least one recipient was suspicious and replied, in effect, "what is this?". Trust in the sender still won.

Key insight. Spam filters and link-reputation services mostly judge the *content* or the *link*, but here the sender was a real, long-standing account. Detection has to look at behaviour (this account suddenly sends the same money lure to many people) and context (the link imitates a brand; the page asks for a password while claiming to be that brand), before and after the click.

3. Threat model

Stage	Attacker capability	Assumption the defences rely on
Network	Operates or observes an open Wi-Fi network	Cannot obtain a valid certificate for the victim's sites, so interception shows up as certificate errors, DNS anomalies or a changed router
Takeover	Holds a valid session or credentials for a real member	Can send messages indistinguishable at the protocol level from the real user
Lure	Registers cheap or look-alike domains	Domains imitate brand names; many are young
Harvest	Clones a brand's sign-in page	Page title or heading claims the brand; the domain does not belong to it
Persistence	Creates new accounts at will	New accounts are young; names often imitate the removed one

Out of scope: malware on the user's computer, a compromised chat platform, and reading private DMs, which platform APIs do not allow (§8.5) and the project does not try to work around.

4. Architecture

Each component answers one or more stages of the attack:

Attack stage	What happened	Control	Component
Network	Session or credentials stolen on open Wi-Fi	Certificate-error capture; DNS-over-HTTPS cross-check; router hardware-address change; DNS answers redirected to private addresses	Extension, netguard
Spread	Hijacked account mass-messages a lure	Message scoring; same-message burst detection; multi-reporter escalation; edit re-scoring	TrustGuard
Lure link	Look-alike Steam gift-card URL	Brand look-alike and brand-in-domain detection; masked-link detection; domain age; optional Safe Browsing	Extension, TrustGuard
Credential harvest	Fake Steam sign-in page	Password form whose title or heading claims a brand the domain does not own	Extension
Re-infiltration	Attacker rejoins with a new account	Holds for removed ids, look-alike names, very new accounts, and a post-compromise lockdown; state survives restarts	TrustGuard

All three tools follow one design rule: **keep the logic pure and the platform code thin.**

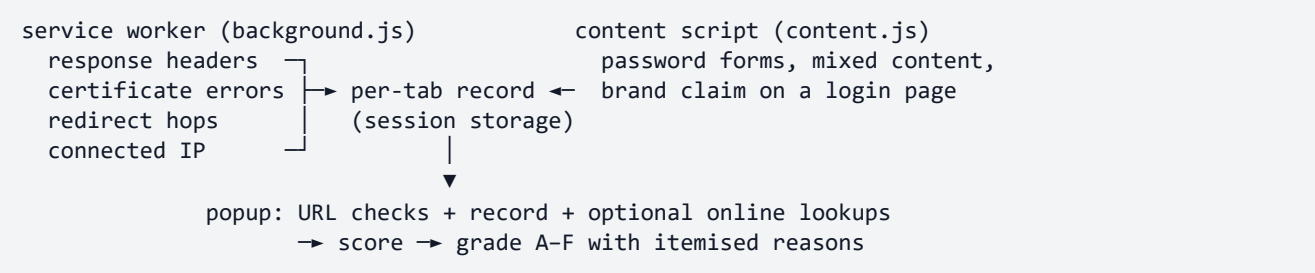
- In the extension, all scoring lives in `lib/core.js`, which makes no `chrome.*` calls. The popup, content script, service worker and Node tests all load it.
- In the bot, `trustguard/core` has no third-party dependencies. Each adapter turns a platform event into a neutral `Message` or `JoinEvent`, calls the engine and acts on the verdict.

- In netguard, parsing and decisions (`checks.py`) are kept apart from the probes that run commands or open connections (`probes.py`).

So each part is testable without a browser, chat workspace or network, every verdict lists its signals, and a new platform needs an adapter, not a new detector.

5. SentinelQA browser extension

5.1 Data flow



Records are kept per tab and deleted when the tab closes. Updates are queued so that two listeners cannot overwrite each other. Findings are shown with `textContent`; the first version inserted the raw URL with `innerHTML`, an injection risk in a security tool, and that was the first fix. The grade stays pinned while findings scroll, and pages that cannot be scanned, such as the new tab, show a grey "?" and no Re-scan button.

5.2 Checks and weights

Level	Check	Points
URL	No HTTPS (local development hosts exempt)	25
URL	Raw IP address as host	30
URL	`@` in the URL (the real host follows it)	20
URL	Punycode (`xn--`) host, possible homoglyphs	25
URL	Abused domain ending (shared rule set)	15
URL	Digits in the domain	5
URL	Pressure words (login, verify, suspended…)	10 each, max 20
URL	URL longer than 75 characters	10
URL	Look-alike of an official brand domain	55
URL	Brand name inside an unrelated domain	35
URL	Brand + reward lure together	15
Page	Each page finding (password over HTTP, mixed content, fake login)	35
Transport	Certificate/TLS error recorded for the tab	70 (always F)
Transport	Each missing security header	8
Transport	Redirect chain of ≥3 hops or ≥3 domains	10
Online	Domain registered < 30 days / < 180 days ago	30 / 10
Online	Server IP not among DNS-over-HTTPS answers	10

Level	Check	Points
Online	Google Safe Browsing match	100

Grades: A < 10, B < 30, C < 50, D < 70, F ≥ 70. The score is capped at 100.

5.3 Brand impersonation

The original URL checks would have scored the incident link low: HTTPS, an ordinary `.com` ending, no IP address, no `@`. They knew nothing about brands. Two rules now compare each domain with a list of commonly impersonated brands and their official domains:

- **Look-alike.** First undo common character swaps (`rn→m`, `vv→w`, `0→o`, `1→l`, `3→e`, `5→s` …). Then compare the domain's main label with each official label by edit distance (the number of single-letter changes needed to turn one into the other). The allowed distance grows with length: 0 for labels under 6 characters, 1 for 6–7, 2 for 8 or more. Without that scaling, ordinary words like `stream` would match `steam`, and a unit test guards that case.
- **Contains.** The brand name appears as a separate word in a domain the brand does not own (`steam-giftcard.xyz`).

When a page has a password field, the content script compares the page's *own* identity (title, `og:site_name`, first heading) with the domain. It ignores "Sign in with Google" buttons and partner logos, which would otherwise raise false alarms on legitimate sites.

5.4 Transport and online checks

The service worker records `ERR_CERT_*` and `ERR_SSL_*` failures for top-level page loads, and keeps them even if the user clicks through the browser's warning, which is when they matter most. It also records redirect hops and the IP address the browser actually connected to.

Online lookups are on by default but run only when the user opens the popup, and only for that site, so browsing history never streams to a third party:

- **RDAP** (the successor to WHOIS, via rdap.org) gives the registration date. Young domains score higher.
- **Cloudflare DNS-over-HTTPS** gives the addresses a trusted resolver returns. If the connected IP is not among them, the score rises by 10. This is a weak signal: CDNs and location-based DNS legitimately differ.
- **Google Safe Browsing** is used only with the organisation's own API key.

Results are cached, and a failed lookup is skipped. An "Online checks" line in the popup shows whether the lookups ran and what they found, or that they are switched off in Settings.

6. Shared rule set

Brand domains, abused domain endings, link shorteners, multi-part suffixes (`co.uk`) and look-alike character pairs live in `shared/rules.json`. `npm run build:rules` generates the extension's copy (`lib/rules.js`) and the bot's (`shared_rules.json` in the Python package), and a test fails if either is stale.

This fixed a real defect. The bot's rule table was copied from the extension and then extended, until the bot recognised Slack impersonation and the `.gift` ending and the extension did not. The same link could

get different verdicts depending on where it was opened. Word lists that belong to one layer stay local: URL pressure words in the extension, greetings and urgency phrases in the bot.

7. netguard: the network stage

The extension sees only web traffic, not the network underneath. netguard covers the Wi-Fi stage from the user's own machine with a few passive checks:

Check	Method	Severity
Router changed	Trust-on-first-use: the router's hardware address, read from the OS ARP cache, is recorded per network on first visit. A different one later on the same network name and gateway IP is how an evil-twin hotspot or ARP spoofing looks to the client	critical
DNS redirected	A probe host resolves to a private address locally but to public addresses over DNS-over-HTTPS	critical
DNS disagreement	Local and DNS-over-HTTPS answers share no /16 block	info; warning only with a certificate failure for the same host
Certificate rejected	A TLS connection to a major site (default: Slack, Microsoft and Google sign-in) fails verification	critical

Exit codes (0 ok, 1 warning, 2 critical) let it run from a login script. It sends no ARP packets, scans nothing and captures no traffic. Active techniques were left out on purpose: they need administrator rights, differ on every operating system, and add little.

Two false warnings on normal networks shaped the DNS check. The load-balanced `login.microsoftonline.com` got different addresses from the same /16 block, so answers are now compared by /16 block. Later `slack.com` raised a WARNING on a home network, because large services give different resolvers different servers, so a lone disagreement is now an info note. Neither change weakens the "private address for a public site" check, which is the one that matters.

8. TrustGuard bot

8.1 Engine

The engine turns a normalised `Message` into a `Verdict`: allow, warn (score ≥ 35) or quarantine (≥ 60), with itemised reasons. Both thresholds are environment variables, so a pilot can start in warn-only mode.

Signal	Points
External link (domain not on the organisation's trusted list)	20
Money / gift / reward lure, or a currency amount	25
Link domain is a look-alike of a brand / uses a brand name	40 / 30
Masked link: visible text shows one domain, the link opens another	30
Short (≤ 25 words), greeting-less message with a link	10
Urgency wording; link shortener; abused domain ending; @everyone / @here	10 each
Sender account under 7 days old (with a link)	10

Signal	Points
Sender with history shares a domain never used before	10
Same message to ≥ 3 destinations within 10 minutes	35

The "no greeting" signal comes from the incident: colleagues usually greet each other, and the lure did not. The greeting list includes the Bangla and Vietnamese forms used in the team.

8.2 Spread detection

A burst is one sender sending the *same* message to several destinations in a short time, as in the incident's mass DM. A fingerprint that ignores case, spacing and numbers decides "same", so "$50" and "$100" versions still match.

8.3 Reports: the private-DM gap

No Discord or Slack bot can read private DMs between two users, and the incident spread through exactly those. So user reports are the bot's window into DMs: a recipient forwards the DM to the bot (Discord) or uses *Share message* (Slack), and gets a verdict and recovery steps back. When two *independent* people report the same sender within 30 minutes, moderators get a "possible compromised account" alert; one person reporting twice does not count.

8.4 Re-verification

A join is held for an admin if any of these is true: the user id was removed or marked compromised before; the name closely resembles a removed account (after Unicode normalisation and the same look-alike swaps used for domains); the account is under 7 days old; or the community is in the 72-hour lockdown that follows marking an account compromised. An admin releases the hold only after checking identity by call or in person.

This control was missing in the incident, so it must survive a restart. Removals, approvals and the lockdown are saved to SQLite: ids, names and timestamps only, never message text, which a test confirms.

8.5 Adapters and platform reality

Platform	Status	What the bot can see and do
Discord	Implemented; tested live	Server channels (Message Content intent) and DMs to the bot. Quarantine deletes the message, times the sender out for one hour and alerts moderators. The alert reports delete and timeout failures separately (no bot can time out the server owner). A hold role restricts held joins. Bans are recorded automatically
Slack	Built and unit-tested with a fake client; not yet tested live. Planned office deployment	Channels it is invited to and DMs to the bot. Bot tokens cannot delete others' messages or deactivate accounts, so quarantine is a threaded warning plus a moderator alert. Deactivations are recorded automatically. Joins to protected channels (including Slack Connect guests) are checked, and held people can optionally be removed. Installs from a manifest with minimal scopes over Socket Mode (no inbound endpoint)
Teams	Future work	Bot Framework
Telegram, WhatsApp	Out of scope	Bots see only messages sent to them / closed API

Both adapters re-score edited messages, so nobody can turn a harmless message into a lure after people have read it. Links quoted in alerts are defanged, made unclickable (`hxxps://evil[.]example`). Slack links bare domains itself, so there the reason text is defanged too; a test found that, not an assumption.

The extension complements the bots: it works wherever a link opens in a browser. The bots protect only where they are invited.

9. Privacy

Component	Reads	Stores	Sends out
Extension	URL, headers and page structure of the active tab	Per-tab results in session storage (cleared on tab close); cached lookup results	Only on popup open, for that site, when online checks are on (default): hostname to rdap.org and Cloudflare; full URL to Safe Browsing if a key is set. Can be switched off
TrustGuard	Messages in channels it was added to; DMs sent to it	Removed and compromised ids, names, approvals, lockdown end (SQLite). Burst and report counters in memory only	Alerts inside the chat workspace; quarantine alerts quote up to 500 characters with links defanged
netguard	OS ARP, route and Wi-Fi tables	Hashed network id, router hardware address and date, on the user's machine	TLS connections and DoH lookups for three fixed probe hosts (DoH optional)

No component logs conversations, records passwords or form contents, or sends browsing history anywhere, and test data is fictional. Before rollout, the guide (`docs/rollout-guide.md`) requires a security review, notice to colleagues (with counsel confirming the lawful basis), and approval of the third-party lookup services.

10. Worked examples

These results come from running the current code.

The incident link in the extension. `https://steamcommunnity.com/gift-card/pay/50` scores 70, grade **F**: look-alike of `steamcommunity.com` (55) plus the brand-and-reward-lure combination (15). The URL-only first version would have graded it A: HTTPS on a `.com` domain. A variant, `https://steam-giftcard.xyz/claim`, scores 65 (D): abused ending (15), brand in an unrelated domain (35) and lure (15). The real `https://store.steampowered.com/` scores 0 (A).

The fake sign-in page. The bundled test page, served over HTTP from a non-local name, reaches 100 (F) by combining no HTTPS, the brand name in the domain, the reward lure, the password-over-HTTP page finding, the fake-login page finding and missing headers.

The incident message in the bot. The bare text `steamcommunnity.com/gift-card/pay/50` scores **95** and is quarantined: external link (20), money lure (25), no greeting (10), brand look-alike (40). Sent by one sender to three DMs within three minutes, the same lure hits the 100 cap with the burst signal (35). Two harmless messages score 0: "hey, standup moved to 10:30 today" and "Hello! Steam summer sale starts tomorrow, anyone want to split a bundle?" The second mentions the brand but has a greeting and no link.

Re-infiltration. After `!tg compromised` on a fictional account `rahim.dev`, a new account named `rahim_dev` that joins five days later is held because its name is a look-alike, even though the 72-hour lockdown has

ended. During the lockdown, every new join is held. Both holds still apply after the bot restarts.

11. Evaluation

Suite	Tests	What is covered
Extension (Node)	26	URL heuristics (clean site, HTTP, raw IP, `@`, punycode, abused ending, keyword cap, long URL), the incident link → F, false-positive guards (`stream.com`, official subdomains), suffix handling, record merging (headers, page findings, certificate errors, redirects), online signals and their summary line, rule-copy parity
TrustGuard (Python)	47	Six fictional lure messages flagged and six benign ones passed; incident link quarantined; burst and window expiry; reports and escalation; masked links; re-verification incl. look-alike rejoin and lockdown; persistence across restart with no message text stored; settings file handling; Slack adapter (reports, commands, edits, channel joins) and Discord adapter (quarantine, owner who cannot be timed out, edits, forwarded messages) against fake clients
netguard (Python)	10	Parsers on captured Windows, macOS and Linux output; router change, DNS and certificate decisions; an end-to-end first-visit-then-swapped-router run

All 83 tests pass. CI runs every suite on each push to `main` and on every pull request, netguard on all three operating systems. For release 1.1.0, everything except Slack was also tested by hand on Windows. In Chrome, the extension caught the fake sign-in page and a bad certificate (both F). netguard reported a home network OK and flagged an expired certificate as critical. On a Discord test server, the bot ignored normal messages; deleted the lure, warned the channel and sent moderators a defanged alert; and caught a masked link, an edit into a lure, and a lure sent to it by DM. The Slack adapter was driven through the real Bolt 1.30 dispatcher but has never run in a live workspace. That live test is the next step (§13).

What this evaluation is not. It is functional testing on a small fictional set modelled on one incident, the same set the weights were tuned on. With no labelled real messages or URLs, precision (how many alerts are real), recall (how many lures are caught) and false-positive rates are unknown. The office pilot (§13) is designed to produce that data.

12. Limitations and future work

Area	Current limitation	Future direction
Risk scoring	Fixed weights; no evasion handling, class-imbalance treatment or measured false-positive rate	ML-based scoring with evasion testing on labelled data
Bot anomaly detection	Public keyword and rate rules can be written around (add a greeting, avoid lure words)	Per-sender statistical baselines; formal threat model
Token theft on Wi-Fi	Symptoms detected (certificate errors, DNS redirection, router change), not the theft	Protocol-level analysis of session handling
Fake login detection	Brand list + form heuristics	Auth-flow analysis (OAuth/SAML failure modes)
Re-verification	Admin approval after an out-of-band check; pragmatic, not formally derived	Formal access-control / zero-trust identity assurance
User behaviour	People click despite doubt	Warning UX design and evaluation
Base rates	With few true phishing messages among many benign ones, even a low false-positive rate means most alerts are false	Threshold selection grounded in measured rates

13. Deployment

The office rollout (`docs/rollout-guide.md`) runs in this order:

1. **Approvals:** security review and change record, notice to colleagues, and approval of the third-party lookup services.
2. **Extension:** a pilot in developer mode, then force-installed through Chrome Enterprise policy.
3. **Bot:** the live Slack test in `docs/pilot-guide.md`, in a separate workspace, and fixes. Then the Slack bot is installed from the manifest, with a private moderators channel and the persistent state file, starting in warn-only mode by raising the quarantine threshold.
4. **netguard:** offered to colleagues who travel.

Moderators marking each pilot alert as a true or false positive will give the first real measurement of the heuristics.

Appendix: reproducing the results

```
npm test
```

```
cd trustguard && python -m unittest discover -s tests -v
```

```
cd netguard && python -m unittest discover -s tests -v
```